



PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

MVP - Data Engineering

Fatores de influência sobre pedidos de delivery em aplicativos de entrega online

Victor Gioia Di Martino

Email: vgioiadimartino@gmail.com

GitHub:

Sprint: Engenharia de Dados

Rio de Janeiro, Setembro de 2026.

Sumário

Contexto de Negócios e Perguntas	4
Contexto	4
Objetivo do Projeto	4
Perguntas de Negócio	4
Fonte de Dados	4
Estrutura dos Dados Brutos	5
Licença dos Dados	6
Carga dos Dados.....	6
Modelagem e Catálogo de Dados	7
Bronze	7
Silver	7
Gold	9
Pipeline de Dados	10
Qualidade de Dados.....	11
Problemas Identificados	12
Soluções Aplicadas	12
Análise de Dados	14
Avaliação de Desempenho do Modelo	16
Autoavaliação	18

Lista de Figuras

Figura 1 – PROCESSO DE INGESTÃO DE CRIAÇÃO DE CAMADA BRONZE	6
Figura 2 - CAMADA BRONZE	7
Figura 3 – TRATAMENTO PARA CAMADA SILVER	9
Figura 4 – PADRONIZAÇÃO DE TEXTO	12
Figura 5 – CRIAÇÃO DE LABELS	13
Figura 6 – ENGENHARIA DE FEATURES.....	13
Figura 7 – CLUSTERIZAÇÃO REGIONAL	14
Figura 8 – TREINAMENTO DO MODELO DE OUTPUT	14
Figura 9 – TREINAMENTO DO MODELO DE FEEDBACK	15
Figura 10 – RANKING DE INFLUÊNCIA DE CARACTERÍSTICAS	16
Figura 11 – CÁLCULO AUC	17
Figura 12 – COMPARAÇÃO REAL X PREVISTO.....	17

Figura 13 – Preparação de Matriz de confusão	18
Figura 14 – MATRIZ DE CONFUSÃO	18

Contexto de Negócios e Perguntas

Contexto

O crescimento dos aplicativos de delivery transformou a forma como consumidores interagem com restaurantes e estabelecimentos comerciais. Atualmente, essas plataformas atuam como intermediadoras entre oferta e demanda, registrando uma grande quantidade de informações sobre o perfil dos usuários e seus hábitos de consumo.

Nesse cenário, compreender os fatores que influenciam a decisão de um cliente realizar um pedido torna-se uma vantagem estratégica. Esse conhecimento permite direcionar campanhas comerciais, personalizar ofertas e melhorar a experiência dos usuários, contribuindo para o aumento das taxas de conversão e satisfação dos clientes.

Objetivo do Projeto

O objetivo deste trabalho é identificar quais características dos consumidores possuem maior influência sobre a realização de pedidos de delivery e sobre a probabilidade de geração de avaliações positivas, utilizando técnicas de engenharia de dados e aprendizado de máquina.

Perguntas de Negócio

O projeto foi desenvolvido para responder às seguintes perguntas:

- Quais características dos consumidores mais influenciam a realização de um pedido de delivery?
- Existe relação entre fatores demográficos e a probabilidade de conversão em pedido?
- Quais fatores estão associados a avaliações positivas dos clientes?
- A localização geográfica influencia o comportamento de consumo?
- Clientes recorrentes apresentam padrões diferentes dos novos clientes?

Fonte de Dados

Foi utilizada uma base pública disponibilizada na plataforma Kaggle contendo informações de consumidores de serviços de delivery na Índia.

A base é composta por registros individuais de consumidores e contém atributos relacionados a:

- Perfil demográfico;

- Faixa de renda;
- Escolaridade;
- Ocupação;
- Localização geográfica;
- Tipo de cliente;
- Resultado do pedido;
- Feedback fornecido pelo consumidor.

Estrutura dos Dados Brutos

A tabela original contém as seguintes colunas:

Campo	Descrição
Age	Idade do consumidor
Gender	Gênero
Marital Status	Estado civil
Occupation	Ocupação
Monthly Income	Faixa de renda
Educational Qualifications	Escolaridade
Family size	Tamanho da família
Customer Type	Tipo de cliente
Latitude	Latitude
Longitude	Longitude
Pin code	Código postal
Output	Realização do pedido
Feedback	Avaliação do consumidor

Licença dos Dados

Os dados foram obtidos em uma fonte pública destinada a fins educacionais e de análise. O trabalho não realiza qualquer identificação individual de consumidores, utilizando exclusivamente informações anonimizadas presentes na base original.

Carga dos Dados

O processo foi iniciado com a importação do arquivo CSV para o ambiente Databricks.

A carga foi realizada por meio das funcionalidades nativas da plataforma, permitindo que o arquivo fosse armazenado no Data Lake associado ao workspace. Em seguida, os dados foram lidos utilizando PySpark e persistidos em formato Delta Lake.

A utilização do formato Delta oferece vantagens como:

- Controle transacional;
- Melhor desempenho de leitura;
- Versionamento dos dados;
- Maior confiabilidade no processamento.

Toda a lógica de ingestão foi desenvolvida no script:



```
df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("/Volumes/workspace/delivery/delivery_data/online_food_delivery_dataset.csv")
)

from pyspark.sql.functions import col

# Sanitize column names: replace spaces with underscores
for old_name in df.columns:
    new_name = old_name.replace(" ", "_")
    if old_name != new_name:
        df = df.withColumnRenamed(old_name, new_name)

df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("delivery.bronze_online_delivery")
```

Figura 1 – PROCESSO DE INGESTÃO DE CRIAÇÃO DE CAMADA BRONZE

Modelagem e Catálogo de Dados

A modelagem foi concebida para suportar tanto atividades analíticas quanto o treinamento de modelos de Machine Learning.

A organização dos dados seguiu uma arquitetura em camadas:

- Bronze
- Silver
- Gold

Bronze

Armazena os dados exatamente como foram recebidos da fonte original.

```
display(spark.table("delivery.bronze_online_delivery").limit(20))
```

	Age	Gender	Marital_Status	Occupation	Monthly_Income	Educational_Qualifications	Family_size
1	20	Female	Single	Student	No Income	Post Graduate	
2	24	Female	Single	Student	Below Rs.10000	Graduate	
3	22	Male	Single	Student	Below Rs.10000	Post Graduate	
4	22	Female	Single	Student	No Income	Graduate	
5	22	Male	Single	Student	Below Rs.10000	Post Graduate	
6	27	Female	Married	Employee	More than 50000	Post Graduate	
7	22	Male	Single	Student	No Income	Graduate	
8	24	Female	Single	Student	No Income	Post Graduate	
9	23	Female	Single	Student	No Income	Post Graduate	
10	23	Female	Single	Student	No Income	Post Graduate	
11	22	Female	Single	Student	No Income	Post Graduate	
12	23	Male	Single	Student	Below Rs.10000	Post Graduate	
13	23	Male	Single	Student	No Income	Post Graduate	
14	21	Male	Single	Student	No Income	Graduate	
15							

20 rows | 22.163s runtime | Refreshed 34m ago

Figura 2 - CAMADA BRONZE

Silver

Contém os dados tratados e padronizados.

Transformações aplicadas:

- Padronização de nomes de colunas;
- Conversão de tipos;
- Criação das variáveis de classificação.

Catálogo de Dados

A Tabela a seguir apresenta o catálogo simplificado dos principais atributos utilizados na solução.

Campo	Tipo	Descrição
age	Integer	Idade do consumidor
gender	String	Gênero informado pelo consumidor
marital_status	String	Estado civil
occupation	String	Ocupação profissional
monthly_income	String	Faixa de renda mensal
educational_qualifications	String	Nível educacional
family_size	Integer	Quantidade de pessoas na família
customer_type	String	Tipo de cliente
latitude	Double	Latitude de localização
longitude	Double	Longitude de localização
pin_code	String	Código postal
age_group	String	Faixa etária derivada
big_family	Integer	Indicador de família numerosa
is_loyal	Integer	Indicador de cliente recorrente
region_cluster	Integer	Cluster geográfico gerado pelo K-Means
label_output	Integer	Variável-alvo para realização de pedido
label_feedback	Integer	Variável-alvo para feedback positivo

```
11:16 PM (1s) 9
from pyspark.sql.functions import col

df = spark.table("delivery.bronze_online_delivery")

for c in df.columns:
    df = df.withColumnRenamed(
        c,
        c.strip()
            .replace(" ", "_")
            .replace(".", "")
            .lower()
    )

> df: pyspark.sql.connect.dataframe.DataFrame = [Age: integer, Gender: string ... 12 more fields]
```

Figura 3 – TRATAMENTO PARA CAMADA SILVER

Gold

A camada Gold foi projetada para suportar diretamente os modelos de Machine Learning. Nessa etapa foram consolidadas todas as variáveis tratadas e criadas as features finais utilizadas no treinamento.

Além dos atributos da camada Silver, foi incorporada uma variável geográfica derivada da aplicação do algoritmo K-Means sobre as coordenadas de latitude e longitude.

Essa nova variável, denominada **region_cluster**, permite representar regiões geográficas com comportamentos semelhantes de consumo, agregando informação espacial ao modelo preditivo.

Catálogo de Dados

A Tabela a seguir apresenta o catálogo simplificado dos principais atributos utilizados na solução.

Campo	Tipo	Descrição
age	Integer	Idade do consumidor
gender	String	Gênero informado pelo consumidor
marital_status	String	Estado civil
occupation	String	Ocupação profissional
monthly_income	String	Faixa de renda mensal
educational_qualifications	String	Nível educacional
family_size	Integer	Quantidade de pessoas na família

customer_type	String	Tipo de cliente
latitude	Double	Latitude de localização
longitude	Double	Longitude de localização
pin_code	String	Código postal
age_group	String	Faixa etária derivada
big_family	Integer	Indicador de família numerosa
is_loyal	Integer	Indicador de cliente recorrente
region_cluster	Integer	Cluster geográfico gerado pelo K-Means
label_output	Integer	Variável-alvo para realização de pedido
label_feedback	Integer	Variável-alvo para feedback positivo

Pipeline de Dados

O pipeline de dados foi desenvolvido integralmente em um único notebook Databricks, reunindo todas as etapas do processo analítico, desde a ingestão dos dados até o treinamento dos modelos de Machine Learning. Essa abordagem foi adotada por se tratar de um projeto acadêmico e de caráter exploratório, permitindo uma visualização sequencial de todas as transformações executadas sobre os dados.

Embora ambientes produtivos frequentemente utilizem múltiplos notebooks ou ferramentas de orquestração para segmentar as etapas do processamento, a centralização em um único notebook facilitou o acompanhamento do fluxo completo e a documentação das atividades realizadas.

O processo foi organizado em blocos lógicos, cada um representando uma etapa específica do pipeline:

1. Importação dos dados
2. Persistência da camada Bronze
3. Tratamento e padronização dos dados

4. Persistência da camada Silver
5. Engenharia de atributos
6. Clusterização geográfica (K-Means)
7. Construção da camada Gold
8. Preparação das features
9. Treinamento dos modelos Random Forest
10. Avaliação e análise de importância das variáveis

Durante a execução do pipeline, as principais etapas de transformação foram persistidas em tabelas Delta Lake. Essa estratégia permitiu armazenar os resultados intermediários do processamento, garantindo rastreabilidade, reaproveitamento dos dados e maior confiabilidade do fluxo analítico.

As tabelas geradas ao longo do processo foram:

- `delivery.bronze_online_delivery`
- `delivery.silver_online_delivery`
- `delivery.gold_consumer_features`

A persistência dos dados em cada etapa permitiu que eventuais ajustes fossem realizados sem a necessidade de reiniciar todo o processo desde a origem dos dados. Além disso, essa abordagem facilita futuras expansões da solução, como a integração com dashboards, novos modelos analíticos ou processos automatizados de atualização de dados.

Todo o código utilizado para implementação do pipeline encontra-se documentado e disponibilizado no repositório GitHub do projeto, permitindo a reprodução integral do processo por outros usuários.

Qualidade de Dados

Antes da modelagem foram realizadas verificações de qualidade para garantir consistência dos dados.

Problemas Identificados

Nomenclatura inconsistente

As colunas possuíam espaços e caracteres especiais.

Exemplo:

- Marital Status
- Family size

Variáveis categóricas em formato textual

Os algoritmos de Machine Learning não trabalham diretamente com variáveis textuais.

Ausência de atributos geográficos consolidados

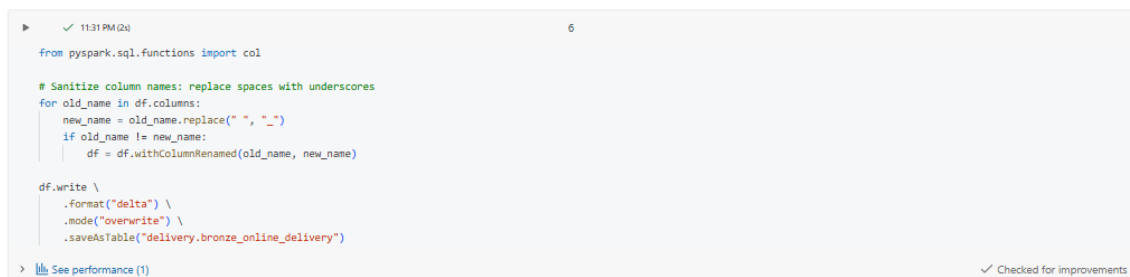
Latitude e longitude estavam disponíveis, mas não representavam regiões de forma interpretável para o negócio.

Soluções Aplicadas

Padronização

Marital Status → marital_status

Family size → family_size



```
from pyspark.sql.functions import col

# Sanitize column names: replace spaces with underscores
for old_name in df.columns:
    new_name = old_name.replace(" ", "_")
    if old_name != new_name:
        df = df.withColumnRenamed(old_name, new_name)

df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("delivery.bronze_online_delivery")
```

Figura 4 – PADRONIZAÇÃO DE TEXTO

Criação de Labels

Output → label_output

Feedback → label_feedback

```
11:16 PM (1s) 12
from pyspark.sql.functions import when

df = df.withColumn(
    "label_output",
    when(col("output")=="Yes",1).otherwise(0)
)

> df: pyspark.sql.connect.dataframe.DataFrame = [age: integer, gender: string ... 12 more fields]
```

```
11:16 PM (<1s) 13
df = df.withColumn(
    "label_feedback",
    when(col("feedback").contains("Positive"),1)
    .otherwise(0)
)

> df: pyspark.sql.connect.dataframe.DataFrame = [age: integer, gender: string ... 13 more fields]
```

Figura 5 – CRIAÇÃO DE LABELS

Engenharia de Features

Foram criadas as variáveis:

- age_group
- big_family
- is_loyal

```
11:31 PM (<1s) 17
# Faixa etária
df = df.withColumn(
    "age_group",
    when(col("age") < 25,"young")
    .when(col("age") < 30,"adult")
    .otherwise("senior")
)

> df: pyspark.sql.connect.dataframe.DataFrame = [age: integer, gender: string ... 14 more fields]
```

```
11:31 PM (<1s) 18
# Tamanho da família
df = df.withColumn(
    "big_family",
    when(col("family_size") >= 5,1)
    .otherwise(0)
)

> df: pyspark.sql.connect.dataframe.DataFrame = [age: integer, gender: string ... 15 more fields]
```

```
11:31 PM (<1s) 19
# Cliente recorrente
df = df.withColumn(
    "is_loyal",
    when(col("customer_type")=="Frequent",1)
    .otherwise(0)
)

> df: pyspark.sql.connect.dataframe.DataFrame = [age: integer, gender: string ... 16 more fields]
```

Figura 6 – ENGENHARIA DE FEATURES

Clusterização Geográfica

Utilizando K-Means foram criados agrupamentos geográficos representados pela variável:

- region_cluster

Essa transformação permitiu introduzir contexto regional no modelo.

```
11:31 PM (<1s) 22
from pyspark.ml.clustering import KMeans

11:31 PM (4s) 23
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans

# Assemble geo features into a single vector column
assembler = VectorAssembler(
    inputCols=["latitude", "longitude", "pin_code"],
    outputCol="geo_features"
)
df = assembler.transform(df)

# Train KMeans clustering model (k=5 as default)
kmeans = KMeans(k=5, seed=42, featuresCol="geo_features", predictionCol="region_cluster")
model = kmeans.fit(df)

# Add cluster prediction column
df = model.transform(df)

# Drop the intermediate vector column
df = df.drop("geo_features")

> df: pyspark.sql.connect.dataframe.DataFrame = [age: integer, gender: string ... 17 more fields]
```

Figura 7 – CLUSTERIZAÇÃO REGIONAL

Análise de Dados

Após a preparação dos dados, foram treinados modelos Random Forest para identificar os fatores associados ao comportamento dos consumidores.

Dois problemas foram analisados:

Modelo 1

Previsão de realização do pedido.

Variável alvo: label_output

```
10:13 PM (11s) 35
#Treinamento do modelo de output
from pyspark.ml.classification import RandomForestClassifier

rf_output = RandomForestClassifier(
    labelCol="label_output",
    featuresCol="features",
    numTrees=300,
    maxDepth=8,
    seed=42
)

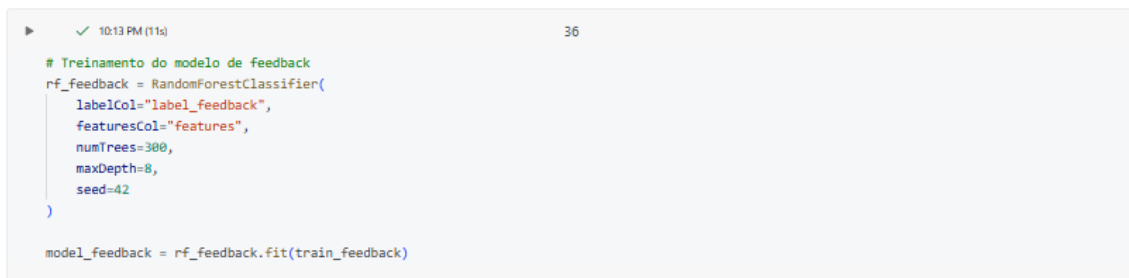
model_output = rf_output.fit(train_output)
```

Figura 8 – TREINAMENTO DO MODELO DE OUTPUT

Modelo 2

Previsão de feedback positivo.

Variável alvo: label_feedback



```
10:13 PM (11s) 36
# Treinamento do modelo de feedback
rf_feedback = RandomForestClassifier(
    labelCol="label_feedback",
    featuresCol="features",
    numTrees=300,
    maxDepth=8,
    seed=42
)

model_feedback = rf_feedback.fit(train_feedback)
```

Figura 9 – TREINAMENTO DO MODELO DE FEEDBACK

Após a preparação dos dados e o treinamento do modelo Random Forest, foi realizada uma análise da importância das variáveis com o objetivo de identificar quais características dos consumidores mais influenciam a realização de pedidos de delivery.

O modelo gerou um ranking de importância das variáveis utilizadas na previsão, permitindo quantificar a contribuição relativa de cada atributo para a tomada de decisão dos consumidores.

Os resultados indicaram que a idade do consumidor foi o fator de maior influência no modelo, apresentando uma importância de aproximadamente 19,2%. Esse resultado sugere que diferentes faixas etárias apresentam padrões distintos de comportamento em relação ao uso de serviços de delivery.

Em seguida, destacou-se o atributo `family_size`, responsável por aproximadamente 8,9% da capacidade explicativa do modelo. Esse resultado indica que o tamanho da família pode influenciar a necessidade e a frequência de utilização de serviços de entrega de alimentos.

Entre as variáveis categóricas, a ocupação profissional, as faixas etárias derivadas (`age_group`) e o tipo de cliente (`customer_type`) apresentaram relevância semelhante, demonstrando que fatores relacionados ao perfil socioeconômico e ao histórico de relacionamento com a plataforma também contribuem para a decisão de realizar um pedido.

A análise também mostrou influência moderada das variáveis de gênero, faixa de renda e estado civil, enquanto características como nível educacional, indicador de fidelidade e família numerosa apresentaram menor impacto relativo no modelo.

```

# Ranking de influência dos atributos

importance_df = spark.createDataFrame(
    zip(feature_names,
        [float(x) for x in importances]),
    ["feature", "importance"]
)

display(
    importance_df.orderBy(
        importance_df.importance.desc()
    )
)

```

importance_df: pyspark.sql.connect.dataframe.DataFrame = [feature: string, importance: double]

	feature	importance
1	age	0.1920528396420566
2	family_size	0.08908490809389362
3	occupation_oh	0.04447307482439288
4	age_group_oh	0.0403931757559705...
5	customer_type_oh	0.03742837805258416
6	gender_oh	0.03383339440583732
7	monthly_income_oh	0.0333863288392806...
8	marital_status_oh	0.0280236282769724...
9	big_family	0.0222866670812309...
10	is_loyal	0.0214949708972239...
11	educational_qualifications_oh	0.0179103776284114...

11 rows | 0.801s runtime | Refreshed 23h ago

Figura 10 – RANKING DE INFLUÊNCIA DE CARACTERÍSTICAS

Os resultados obtidos permitiram responder às perguntas de negócio definidas no início do projeto. Foi possível verificar que características demográficas, especialmente idade e composição familiar, possuem influência significativa sobre a probabilidade de realização de pedidos. Além disso, atributos relacionados ao perfil do consumidor, como ocupação, renda e tipo de cliente, também contribuem para explicar o comportamento observado.

De forma geral, o estudo demonstrou que a decisão de compra em plataformas de delivery é influenciada por uma combinação de fatores demográficos, socioeconômicos e comportamentais. A utilização de técnicas de Machine Learning possibilitou não apenas prever o comportamento dos consumidores, mas também identificar e quantificar a relevância de cada variável no processo de decisão.

Avaliação de Desempenho do Modelo

Para garantir que o modelo apresentou um desempenho satisfatório na previsão dos resultados, foram aplicados diferentes métodos de avaliação sobre os outputs gerados.

AUC

O modelo apresentou uma AUC de 0,81, indicando boa capacidade de distinguir consumidores com maior e menor probabilidade de realizar um pedido. Esse

resultado demonstra que o modelo possui poder discriminatório adequado para identificar padrões associados ao comportamento dos consumidores.

```
10:13 PM (2s) 39
# Calcular AUC para avaliar performance do modelo
from pyspark.ml.evaluation import BinaryClassificationEvaluator

evaluator = BinaryClassificationEvaluator(
    labelCol="label_output",
    metricName="areaUnderROC"
)

auc = evaluator.evaluate(pred_output)

print(f"AUC: {auc:.4f}")

AUC: 0.8056
```

Figura 11 – CÁLCULO AUC

Comparação Real x Previsto

Foi realizada uma comparação entre o resultado real e o resultado previsto pelo modelo, juntamente com a probabilidade associada ao evento de interesse. Os resultados mostram que o modelo acertou a maioria das previsões com elevado grau de confiança, evidenciando boa aderência aos dados analisados e capacidade satisfatória de generalização.

```
10:57 PM (14s) 40
# Comparação Real x Previsto

from pyspark.sql.functions import udf
from pyspark.sql.types import DoubleType

extract_prob = udf(
    lambda v: float(v[1]),
    DoubleType()
)

pred_output = pred_output.withColumn(
    "prob_order",
    extract_prob("probability")
)

display(
    pred_output.select(
        "label_output",
        "prediction",
        "prob_order"
    )
)
```

pred_output: pyspark.sql.connect.dataframe.DataFrame

	1.0 label output	1.2 prediction	1.2 prob_order
1	1	1	0.9309554418454151
2	1	1	0.9584851452325022
3	1	1	0.9560178564901786
4	1	1	0.9058017852600909
5	1	1	0.872532120070344
6	0	1	0.862565004446065
7	1	1	0.936775935107718
8	0	0	0.107268963528316...
9	1	1	0.955745662101808
10	1	1	0.955745662101808
11	1	1	0.954992079613223
12	1	1	0.954992079613223
13	1	1	0.954992079613223
14	0	1	0.5713014370525272

Figura 12 – COMPARAÇÃO REAL X PREVISTO

Matriz de Confusão

A matriz de confusão demonstra que o modelo foi capaz de classificar corretamente a maior parte dos consumidores, apresentando uma quantidade reduzida de falsos positivos e falsos negativos. Dessa forma, observa-se um bom equilíbrio entre os acertos e erros de classificação, reforçando a confiabilidade dos resultados obtidos.

```
pdf = pred_output.select(
    "label_output",
    "prediction"
).toPandas()

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(
    pdf["label_output"],
    pdf["prediction"]
)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Previsto")
plt.ylabel("Real")
plt.title("Matriz de Confusão")
plt.show()
```

Figura 13 – Preparação de Matriz de confusão

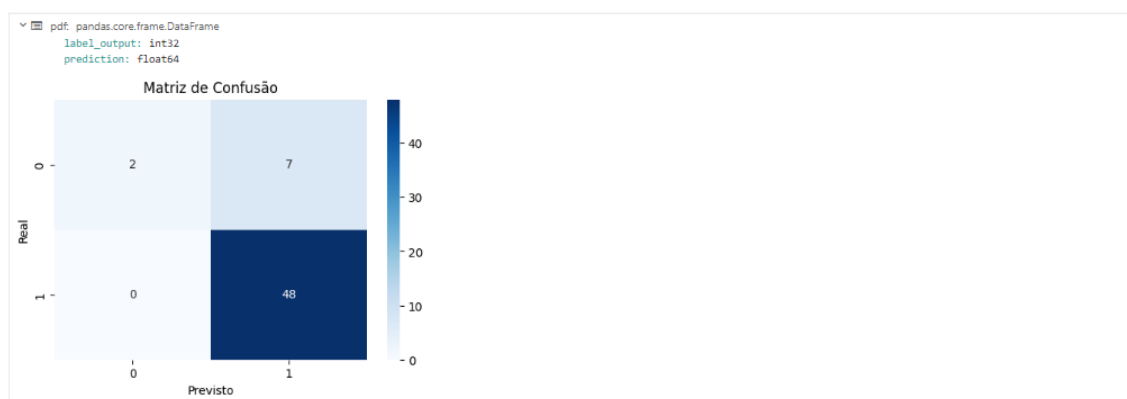


Figura 14 – MATRIZ DE CONFUSÃO

Autoavaliação

Os objetivos definidos no início do projeto foram atingidos com sucesso. Foi possível construir um pipeline completo de dados, desde a ingestão da base bruta até a geração de modelos capazes de identificar os fatores mais influentes no comportamento dos consumidores.

Entre os principais aprendizados obtidos destacam-se a utilização prática do Databricks, o desenvolvimento de pipelines utilizando Python e algo em SQL e a aplicação de técnicas de Machine Learning para problemas reais de negócio.

As principais dificuldades encontradas estiveram relacionadas à preparação dos dados para modelagem, à transformação de variáveis categóricas e à construção das etapas de engenharia de atributos que agregassem valor preditivo ao modelo.

Seria interessante aplicar os seguintes passos para uma evolução do projeto:

- incorporar novas fontes de dados;
- utilizar algoritmos mais avançados, como XGBoost;
- implementar análise explicável utilizando SHAP;
- integrar o ambiente ao MLflow para rastreamento de experimentos;
- disponibilizar os resultados em dashboards interativos em alguma plataforma de visualização como o Power BI.

Essas melhorias poderão aumentar a capacidade explicativa da solução e aproximar o projeto de um ambiente produtivo real.